

Definitive Guide to Immutable Cold Storage Architecture: Implementing a Multi-Tenant, Infinite-Incremental Optical Archival System for Linux

1. Executive Summary

The preservation of high-volume, static data assets presents a unique engineering challenge that stands in stark contrast to the requirements of high-availability transactional systems. The objective outlined is the creation of a "Write-Once, Read-Never" archival system for terabytes of Linux-hosted data, utilizing cold offline optical media (Blu-ray and M-Disc) as the storage substrate.

This report details the **Linux Cold Storage Archival Suite (LCSAS)** architecture. This custom workflow leverages **Content-Defined Chunking (CDC)** to achieve storage-agnostic deduplication, ensuring that file moves and renames consume only negligible metadata space rather than duplicating payload storage.

To meet the requirements of **decentralized recovery** and **multi-volume management**, this version of the architecture introduces the **Holographic Metadata Strategy**. This approach utilizes a self-contained **SQLite database** that indexes the location of every encrypted data chunk across the physical media fleet. A copy of this master index is written to every new volume, ensuring that possessing the most recent disc provides a complete map of the entire archive, enabling recovery from any valid combination of surviving media.

2. Problem Space Analysis and Theoretical Constraints

The transition from magnetic storage (Hard Disk Drives, LTO Tape) to optical storage (Blu-ray, M-Disc) for terabyte-scale archives introduces strict physical limitations that software must overcome.

2.1 The Paradox of Write-Once Incrementalism

The primary challenge is the requirement for "incrementals... optimized for moved/renamed files" without "ever refreshing a full backup." Standard backup tools (tar, rsync) fail here; renaming a directory necessitates re-writing the entire data payload to the new medium, which is prohibitively expensive on write-once media.

The solution requires **Content-Defined Chunking (CDC)**. We utilize **Rustic** (a Rust implementation of the Restic format) to fingerprint data streams. If a file is renamed, its chunks remain identical. The system detects these existing chunks and writes **zero** new payload data to the optical disc, only a small metadata reference.

2.2 Cryptographic Compartmentalization on Shared Media

To satisfy the requirement of "different keys for distinct datasets" while "mixing small incrementals... on the same disc," we treat the optical disc as a "dumb" container for encrypted objects.

- **Object-Level Encryption:** Each file chunk is individually encrypted with the key corresponding to its dataset (Family, Personal, Friend) *before* it is written to the staging area.
- **Result:** A single M-Disc can hold packs from three different datasets. A user with the "Friend" key can mount the disc but can only decrypt the chunks belonging to them.

2.3 The "Holographic" Index Requirement

A critical risk in multi-volume offline storage is **Index Fragmentation**. If Disc 1 holds data, and Disc 2 holds the index required to read Disc 1, losing Disc 2 renders Disc 1 useless.

- **Solution:** We implement a **Cumulative Index**. Every time a new disc is burned, it includes a snapshot of the *entire* repository's metadata and a copy of the Master SQLite Catalog.
- **Benefit:** The latest disc acts as a "Recovery Key." It tells you exactly which previous discs are needed to restore a specific snapshot.

3. Architecture: The Split-Horizon & Local Mirror

This architecture separates the repository into two physical domains but maintains a permanent link between them via the Master Index.

3.1 The Three-Tier Storage Topology

1. **Tier 1: The Local Mirror (Hot/Warm)**
 - **Role:** Permanent repository cache.
 - **Content:** Contains *all* data packs and *all* metadata.
 - **Purpose:** Allows for fast "virtual" restores, instant consolidation (creating new M-Discs without fetching old ones), and mirroring to new media types (e.g., LTO tape) in the future.
2. **Tier 2: The Staging Area (Transient)**
 - **Role:** buffer for burn operations.
 - **Content:** Hardlinks to new data packs that have not yet been committed to optical media.
3. **Tier 3: The Cold Archive (Offline)**
 - **Role:** Disaster recovery.
 - **Content:** Optical Discs (BD-R / M-Disc) stored offsite. Each disc contains a subset of data packs but a *superset* of metadata.

3.2 The Master Database Schema (SQLite)

To manage terabytes of data across potentially hundreds of discs, we cannot rely on simple text files. We use a **SQLite database** (archive_master.db) to track the relationship between logical data (Packs) and physical storage (Volumes).

Table: volumes

- volume_id (PK): Integer ID.
- label: Human-readable label (e.g., BD_005, MDISC_001).
- uuid: Unique identifier for the physical disc.
- type: Enum (BDR, MDISC, LTO).
- location: Text (e.g., "Offsite Safe", "Home Shelf").
- created_at: Timestamp.

Table: packs

- pack_id (PK): The SHA256 hash of the encrypted pack (from Rustic).
- size: Size in bytes.

Table: volume_packs (Many-to-Many Relationship)

- *Crucial for the "Recover from any combination" requirement.*
- pack_id: FK to packs.
- volume_id: FK to volumes.
- *Concept:* This allows a single pack (e.g., a common OS file header) to exist on BD_001, BD_002, and MDISC_001. If BD_001 is destroyed, the database knows we can still get that pack from BD_002 or MDISC_001.

Table: snapshots

- snapshot_id: Rustic snapshot ID.
- hostname: Origin of data.
- dataset: (Family, Personal, Friend).
- timestamp: Backup time.

4. Operational Workflow

4.1 Daily Routine: Ingest to Local Mirror

The daily backup job runs rustic backup against the local server storage.

1. **Destination:** The **Local Mirror** (e.g., /mnt/backup_hdd/repo).
2. **Action:** Rustic chunks new files, deduplicates against the Local Mirror, and writes new packs.
3. **Result:** The Local Mirror is always a complete, restore-ready backup repository.

4.2 The "Burn Cycle": Staging and Indexing

Periodically (e.g., monthly or when 25GB of new data accumulates), we migrate data to optical media.

1. **Identify New Packs:**

The Orchestrator Script queries the archive_master.db:

```
SELECT pack_id FROM packs WHERE pack_id NOT IN (SELECT pack_id FROM volume_packs)
```

This returns a list of packs residing on the Local Mirror that have *never* been burned to a disc.

2. **Bin Packing:**

The script selects enough "unburned" packs to fill a BD-R (approx 23GB).

3. **Staging:**

The script hardlinks these selected packs from the Local Mirror to a temporary staging directory structure (e.g., /mnt/staging/data/).

4. **Holographic Metadata Injection:**

The script copies the **entire** metadata directory (/index, /snapshots, /keys, /config) from the Local Mirror to /mnt/staging/.

- Note: Metadata is small. Including the full history on every disc ensures self-sufficiency.

5. **Database Update:**

- Create entry in volumes table (e.g., BD_024).
- Insert mappings into volume_packs for every pack in the staging area.
- Copy the updated archive_master.db into the root of /mnt/staging/.

6. **Burn and Verify:**

Generate ISO -> Add ECC (DVDDisaster) -> Burn to Disc.

Verify integrity. If successful, update the volumes table status to VERIFIED.

4.3 The Consolidation Workflow (Migration)

This addresses the requirement to "consolidate small discs into a single 100GB M-Disc" without fetching them from offsite.

1. **Select Source:** The database identifies that BD_001 through BD_004 contain 95GB of data combined.
 2. **Fetch from Local:** Since we maintain the **Local Mirror**, we do *not* need the physical BD-Rs. The script simply reads the relevant packs from the local HDD.
 3. **Pruning Logic:**
 - Run rustic prune --dry-run on the Local Mirror. Identify "dead" packs (deleted files).
 - Filter the list: Only select packs from BD_001..004 that are *not* dead.
 4. **Burn M-Disc:**
 - Write these active packs to a new staging area.
 - Update DB: Create volume MDISC_001. Map these packs to MDISC_001 in volume_packs.
 5. **Physical Disposal:** You can now destroy BD_001..004. The DB knows their packs are safe on MDISC_001.
-

5. Software Recommendation and Implementation

To implement this, we require a custom "Orchestrator" script to glue rustic, sqlite3, and xorriso together.

5.1 The Orchestrator Script (lcsas-manager.py)

I recommend implementing this in Python for robust string handling and SQLite support.

Key Functions:

- `sync_local()`: Wraps rustic backup.
- `status()`: Queries DB. "You have 45GB of unarchived data (approx 2 BD-Rs)."
- `prepare_burn(media_type="BD25")`:
 - Queries DB for unarchived packs.
 - Fills a "bin" up to capacity.
 - Generates a `burn_manifest.json`.
 - Updates DB transactionally (pending state).
- `finalize_burn(volume_label)`:
 - Called after successful burning.
 - Commits DB changes (sets volume to VERIFIED).
- `generate_restore_plan(snapshot_id)`:
 - Runs rustic restore `--dry-run --json`.
 - Parses output list of needed packs.
 - Queries DB to find which Volumes hold those packs.
 - Outputs instructions: "Insert Disc 4, Disc 8, and Disc 12."

5.2 Database Schema (SQL)

SQL

```
CREATE TABLE volumes (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  label TEXT UNIQUE NOT NULL,    -- e.g. "LCSAS_BD_2023_01"  
  media_type TEXT,              -- "BD25", "MDISC100", "LTO5"  
  location TEXT,                -- "Offsite_A", "Safe_B"  
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE packs (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  sha256 TEXT UNIQUE NOT NULL,   -- The Rustic Pack ID  
  size_bytes INTEGER NOT NULL  
);
```

```

-- Maps Packs to Volumes.
-- A pack can exist on multiple volumes (Mirrors/Consolidations)
CREATE TABLE volume_pack_map (
    volume_id INTEGER,
    pack_id INTEGER,
    PRIMARY KEY (volume_id, pack_id),
    FOREIGN KEY(volume_id) REFERENCES volumes(id),
    FOREIGN KEY(pack_id) REFERENCES packs(id)
);

-- Index of Snapshots (for searching without mounting the repo)
CREATE TABLE snapshots (
    id TEXT PRIMARY KEY,          -- Rustic Snapshot ID
    dataset_key TEXT,             -- "Family", "Work"
    timestamp DATETIME,
    description TEXT
);

```

6. Restoration Scenario (The "Pick List")

When a restore is needed and the local server is dead:

1. **Bootstrap:** Grab the *most recent* disc available.
2. **Load Index:** Copy archive_master.db and the /restic/ metadata directory from that disc to a temporary SSD.
3. **Query:** User asks to restore "Family Photos 2023".
4. **Resolution:**
 - The system checks the metadata to find the specific Pack IDs needed for that file tree.
 - The system queries archive_master.db:


```
SQL
SELECT label, location FROM volumes
JOIN volume_pack_map ON volumes.id = volume_pack_map.volume_id
JOIN packs ON volume_pack_map.pack_id = packs.id
WHERE packs.sha256 IN (...list of needed packs...)
```
5. **Output:** "Please insert Volume BD_005 (Safe A) and MDISC_002 (Offsite)."
6. **Feed:** The user inserts discs. The system copies the specific packs into a local staging directory.
7. **Decrypt:** Once all packs are staged, standard rustic restore is run against the staging directory.

7. Future Proofing: LTO and Cloud

Because we maintain the **Local Mirror** and an abstract database map:

- **LTO Tape:** To add a tape tier, we simply create a job that writes unarchived packs to tape instead of ISO. The `media_type` in the DB becomes LTO5. The logic remains identical.
- **Cloud:** We can define an S3 bucket as a "Volume" in our database. `rcclone` syncs packs to S3, and we record that Pack X exists in Volume: `AWS_Bucket`.

8. Summary of Recommendations

1. **Storage:** Dedicate a large HDD (e.g., 8TB+) on the server as the **Permanent Local Mirror**. Do not delete packs from here after burning; keep them for consolidation and fast recovery.
2. **Software:** Use **Rustic** for the core engine, **SQLite** for the catalog, and **Xorriso** + **DVDDisaster** for the optical media layer.
3. **Workflow:** Adopt the "Holographic" strategy—every disc contains the map to the whole archive.
4. **Consolidation:** Perform consolidation entirely within the Local Mirror, only burning the final sorted result to M-Disc.

This plan meets all your criteria: deduplication for renames (via Rustic), multi-tenant encryption (via object-level keys), infinite incrementals (via CDC), multi-volume tracking (via SQLite), and media consolidation (via Local Mirror).